



Documentation technique

TP Développeur Web et Web Mobile

Léna De Bastos

Traiteur artisanal à Bordeaux — <https://vite-gourmand-old-lagoon-1903.fly.dev>

1. Réflexions technologiques initiales

Le cahier des charges laissait le choix des technologies libre, à condition de justifier ce choix. Pour la partie back, j'ai écarté un framework type Symfony ou Laravel : le projet reste de taille raisonnable (une quinzaine de tables, quatre rôles), et je voulais surtout être sûre de comprendre chaque brique plutôt que de m'appuyer sur des abstractions que je maîtrise encore mal à ce stade.

J'ai donc construit un MVC fait maison en PHP 8.2 : un routeur simple qui fait correspondre des URL à des méthodes de contrôleur, des modèles qui encapsulent les requêtes PDO, et des vues PHP classiques. Composer sert uniquement à gérer l'autoload et le pilote MongoDB, pas à installer un framework complet.

Pour la base de données relationnelle, MySQL s'imposait par habitude (c'est ce qu'on utilise en cours et ce qu'XAMPP propose par défaut) et parce que le modèle de données du projet est très relationnel : commandes, menus, plats, allergènes, avis... des tables avec des clés étrangères partout, exactement ce pour quoi une base relationnelle est faite.

Le choix plus atypique du projet, c'est MongoDB pour les statistiques de l'espace admin. L'idée est venue du besoin d'agréger des chiffres d'affaires par menu et par période sans multiplier les jointures et les GROUP BY complexes côté MySQL. MongoDB n'est utilisé que pour ça : une collection de documents dénormalisés, alimentée en parallèle des commandes MySQL, sur laquelle j'utilise l'aggregation framework pour obtenir directement les chiffres affichés dans les graphiques Chart.js. Le reste de l'application (comptes, commandes, menus) ne touche jamais à MongoDB : ça reste cantonné à cet usage précis, pour ne pas avoir à gérer deux sources de vérité sur les mêmes données.

Côté front, pas de framework JavaScript non plus. Les filtres dynamiques sur la liste des menus et le calcul de prix en temps réel dans le formulaire de commande sont faits en JS vanilla avec des appels fetch vers des endpoints PHP qui renvoient du JSON. Vu la taille des interactions à gérer, un framework comme React ou Vue aurait ajouté de la complexité de build (webpack, npm, transpilation) pour un bénéfice qui ne se justifiait pas ici.

Pour le déploiement, j'ai choisi Docker + fly.io plutôt qu'un hébergement mutualisé classique, pour avoir un environnement de production qui reproduit fidèlement l'environnement de dev (même version de PHP, mêmes extensions) et pour pouvoir versionner la configuration serveur (Dockerfile, fly.toml) au même titre que le code.

2. Configuration de l'environnement de travail

En local

- XAMPP (Apache + MySQL) comme serveur de dev, avec le DocumentRoot pointé vers le dossier public/ du projet plutôt que sur la racine, pour ne jamais exposer les fichiers de config ou les vues brutes.
- PHP 8.2 et Composer installés en dehors d'XAMPP, ajoutés au PATH Windows.
- MongoDB Community Server (installé en local pendant le développement) avec MongoDB Compass pour inspecter les collections, et l'extension PHP mongodb activée dans php.ini.
- VS Code comme éditeur, avec les extensions PHP Intelephense (autocomplétion et navigation dans le code), PHP Debug (Xdebug), et GitLens pour visualiser l'historique et les branches directement dans l'éditeur.
- Un fichier .env (non versionné) pour les identifiants de connexion locaux, sur le modèle d'un .env.example fourni dans le dépôt.

Gestion de version

Le dépôt est hébergé sur GitHub. Le travail est organisé autour de trois niveaux de branches : main ne contient que du code stable et déployable, develop sert de branche d'intégration pour les fonctionnalités en cours, et chaque évolution (une fonctionnalité, une correction, une mise à jour de documentation) passe par une branche feature/nom-de-la-tache créée à partir de develop puis fusionnée dedans une fois terminée.

Une fois develop stabilisée, elle est fusionnée dans main pour préparer un déploiement. Ce découpage évite de pousser du code non testé directement en production et garde un historique de commits lisible, feature par feature.

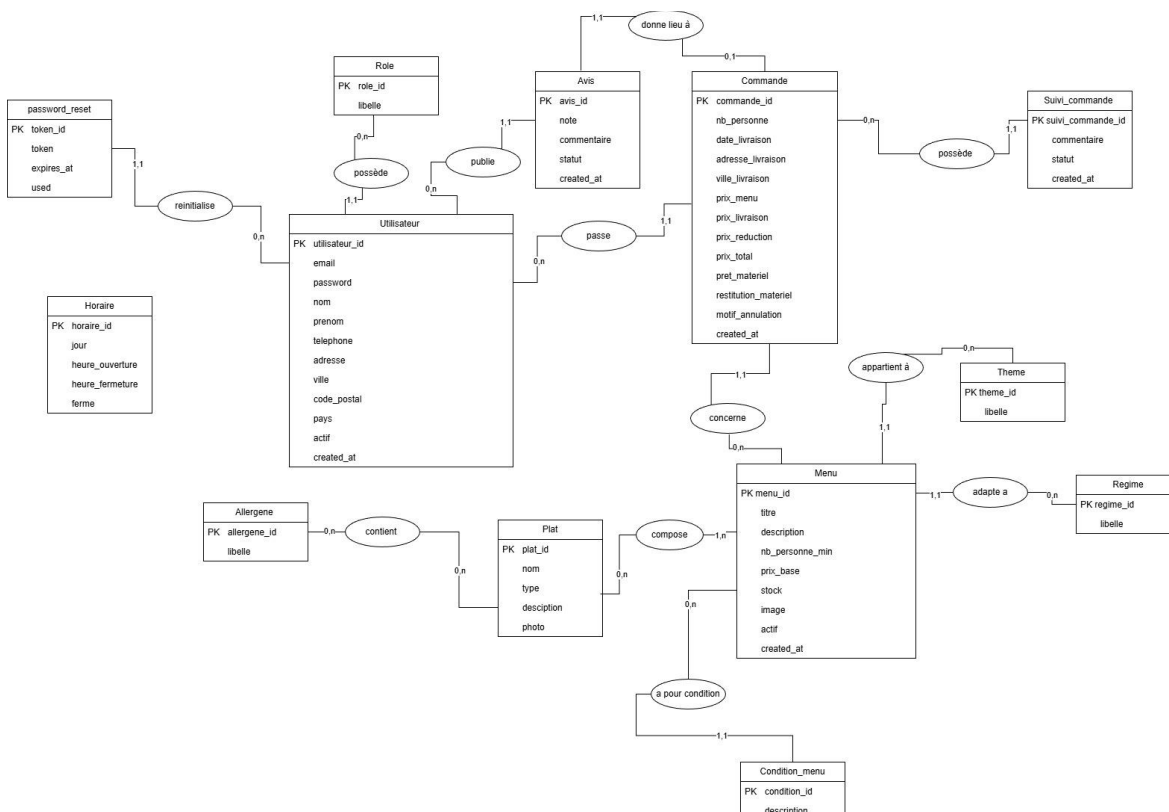
Composer gère les dépendances PHP (essentiellement le pilote MongoDB), avec composer.json et composer.lock versionnés pour que l'installation soit reproductible d'une machine à l'autre.

Base de données

Le schéma MySQL est défini dans database/database.sql (structure + données de démonstration), importable directement via phpMyAdmin ou en ligne de commande. Un script database/migrate.php gère les évolutions de schéma ultérieures (par exemple l'ajout de la table login_attempt pour la limitation des tentatives de connexion), pour ne pas avoir à réimporter tout le schéma à chaque changement, en local comme en production.

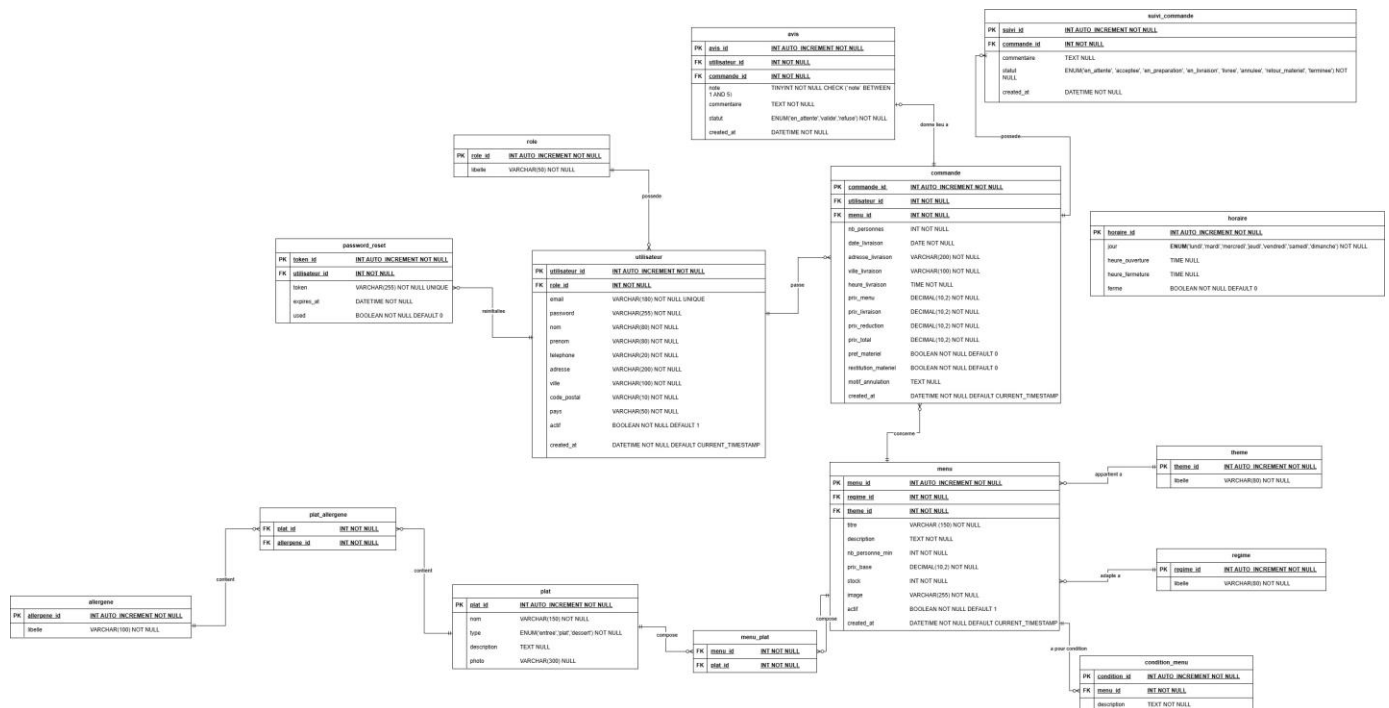
3. Modèle conceptuel de données

Voici le MCD de mon application, fait avec la méthode Merise. On y voit les grandes entités et comment elles sont reliées entre elles, avec les cardinalités. Un utilisateur a un rôle, il passe des commandes et il peut publier des avis. Une commande concerne un seul menu et peut donner lieu à un avis. Un menu appartient à un thème, s'adapte à un régime, se compose de plusieurs plats et a ses propres conditions. Chaque plat peut contenir plusieurs allergènes. J'ai aussi une table pour la réinitialisation du mot de passe et une table pour les horaires. Le suivi des commandes est géré par une entité à part, comme ça je garde tout l'historique des statuts au lieu d'écraser à chaque changement



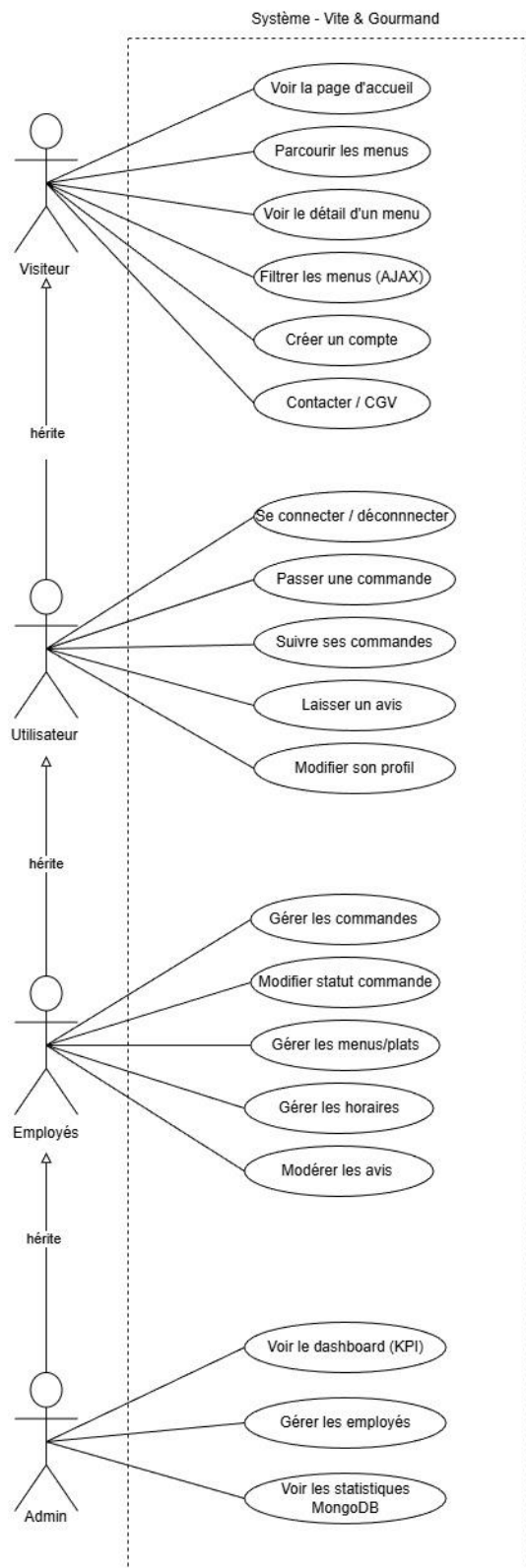
4. Le modèle logique de données

Le MLD, c'est le MCD transformé en vraies tables SQL, avec les types, les clés primaires et les clés étrangères. Les relations plusieurs-à-plusieurs deviennent des tables de liaison : `menu_plat` pour associer les plats aux menus, et `plat_allergene` pour les allergènes. Pour le reste, j'utilise des clés étrangères directes (par exemple une commande garde l'id de l'utilisateur et du menu). J'ai mis quelques règles directement dans le schéma : la note d'avis entre 1 et 5, les statuts en ENUM, l'email et le token en unique, et des valeurs par défaut. C'est ce schéma que j'ai repris tel quel dans `database/database.sql`.

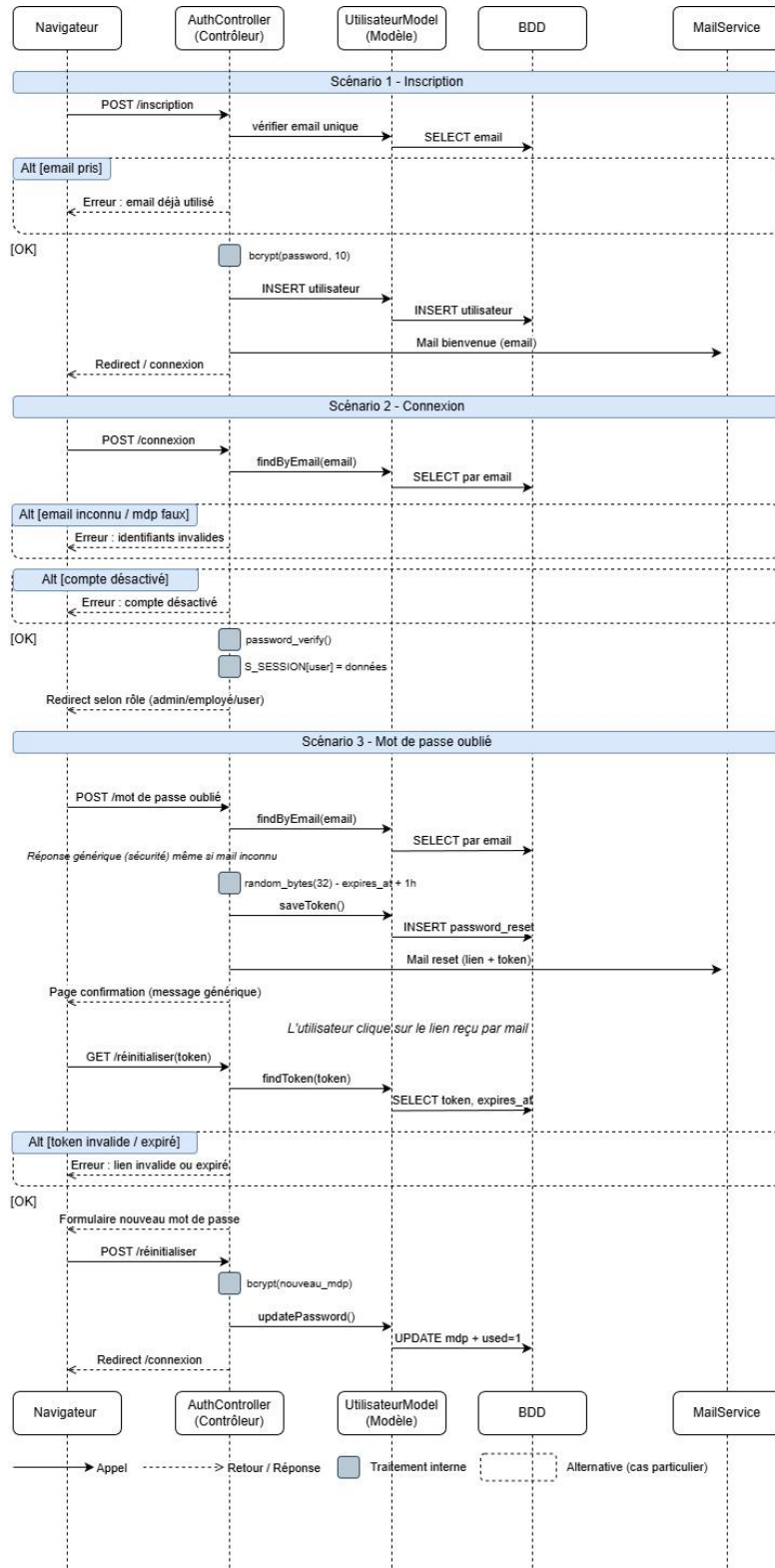


5. Le diagramme de cas d'utilisation

Ce diagramme montre les quatre types d'utilisateurs et ce que chacun peut faire, avec un système de droits qui s'ajoutent. Le visiteur peut voir l'accueil, parcourir et filtrer les menus, en voir le détail, créer un compte et nous contacter. L'utilisateur connecté peut en plus passer commande, suivre ses commandes, laisser un avis et modifier son profil. L'employé peut gérer les commandes, changer leur statut, gérer les menus et les horaires, et modérer les avis. Enfin l'administrateur peut tout faire comme un employé, plus gérer les comptes employés et voir les statistiques. Dans le code je ne l'ai pas fait avec de l'héritage de classes : chaque contrôleur vérifie juste le rôle en session.

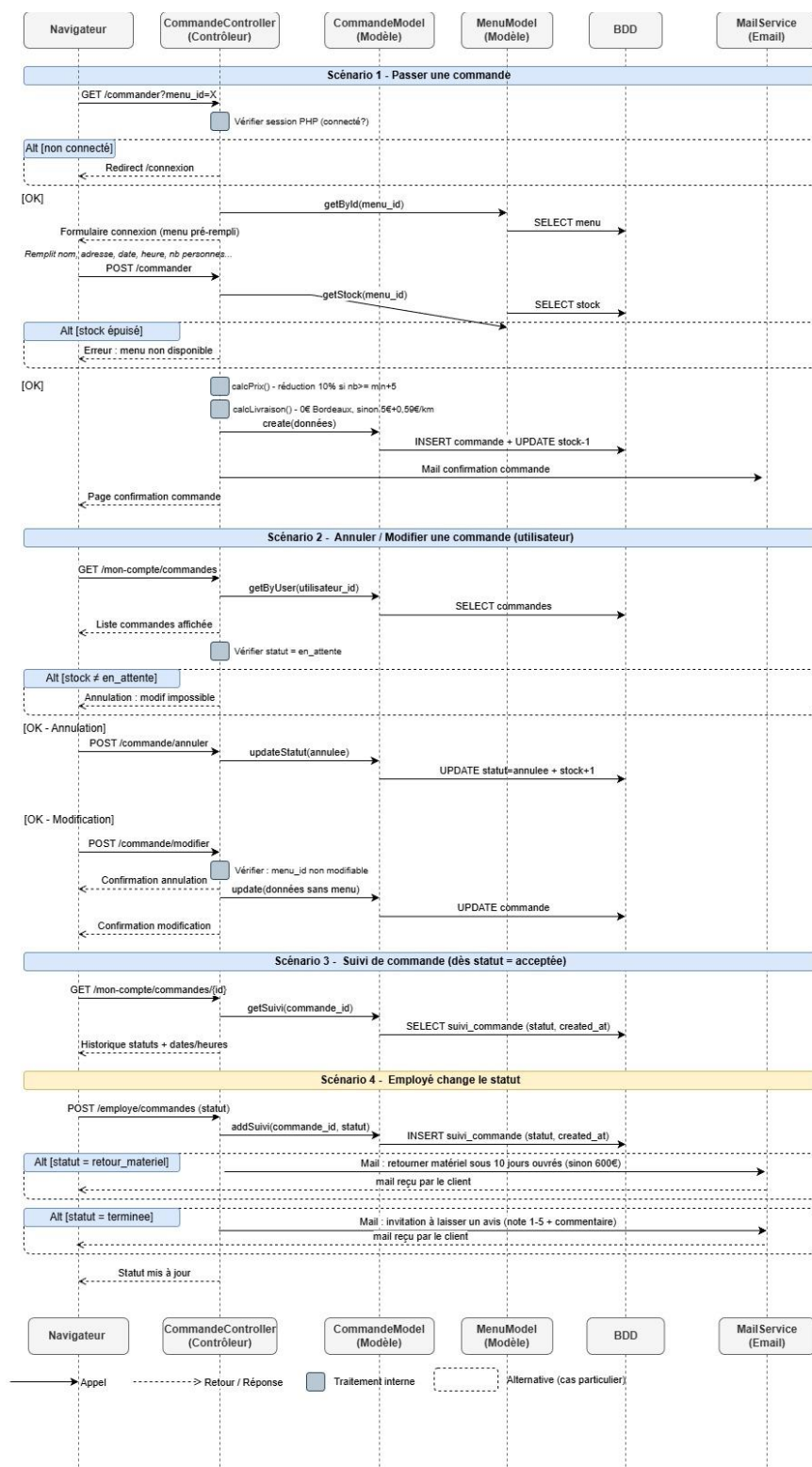


Ce diagramme montre les trois parcours liés au compte : l'inscription, la connexion et le mot de passe oublié. À l'inscription, je vérifie que l'email n'existe pas déjà, je hache le mot de passe, j'enregistre l'utilisateur et j'envoie un mail de bienvenue. À la connexion, je vérifie le mot de passe puis je redirige selon le rôle. Pour le mot de passe oublié, je renvoie toujours le même message que l'email existe ou pas (pour ne pas donner d'info à un attaquant), je génère un token avec `random_bytes(32)` qui expire au bout d'une heure, je l'envoie par mail, et une fois utilisé il n'est plus valable.



7. Diagramme de séquence - Cycle de vie d'une commande

Ici je détaille tout le parcours d'une commande. Quand l'utilisateur commande, je vérifie qu'il est connecté, le formulaire est pré-rempli avec le menu, je vérifie le stock, puis je calcule le prix (avec la réduction de 10% à partir de 5 personnes en plus du minimum) et les frais de livraison (gratuit dans Bordeaux, sinon 5€ + 0,59€/km). Ensuite il reçoit un mail de confirmation. Il peut annuler ou modifier tant que la commande est « en attente », et suivre l'avancement. Côté employé, chaque changement de statut est enregistré dans l'historique, et deux statuts envoient un mail automatique : le retour du matériel (à rendre sous 10 jours ouvrés sinon 600€) et la commande terminée (pour inviter à laisser un avis).



8. Documentation du déploiement

L'application est déployée sur fly.io dans un conteneur Docker construit à partir d'une image PHP 8.2 + Apache. Le choix de Docker permet de garder la même version de PHP et les mêmes extensions (notamment mongodb) entre le poste de dev et la production, ce qui évite les mauvaises surprises du type 'ça marche en local mais pas en prod.

Fichiers de configuration

- Dockerfile : construit l'image (PHP 8.2, Apache, extensions PDO MySQL et mongodb, activation de mod_rewrite, copie du code et installation des dépendances Composer).
- fly.toml : configuration de l'application fly.io (nom, région, port interne, ressources allouées).
- nixpacks.toml : utilisé en complément par fly.io pour certains réglages de build.
- 000-default.conf : configuration Apache définissant public/ comme racine du site et autorisant les .htaccess.

Secrets et variables d'environnement

Aucune valeur sensible ne se trouve dans le dépôt : les secrets de production (DB_HOST, DB_NAME, DB_USER, DB_PASS, DB_PORT, MONGO_URI, APP_URL) sont configurés directement dans les secrets fly.io via la commande fly secrets set, et ne sont donc jamais visibles dans le code source ni dans l'historique Git. Étapes d'un déploiement

- 1. Développement et tests en local sur une branche feature/*, fusionnée dans develop une fois validée.
- 2. Fusion de develop dans main quand un ensemble de fonctionnalités est prêt à partir en production.
- 3. fly deploy reconstruit l'image Docker à partir du Dockerfile et la déploie sur l'infrastructure fly.io.
- 4. Si le déploiement introduit une modification de schéma (par exemple l'ajout d'une table), la migration est lancée manuellement via fly ssh console -a vite-gourmand-old-lagoon-1903 -C "php /var/www/html/database/migrate.php", pour garder le contrôle sur le moment où le schéma de production change.
- 5. Quelques vérifications rapides après déploiement : accès à la page d'accueil, connexion avec un compte de test, affichage des statistiques admin (pour s'assurer que la connexion MongoDB fonctionne bien en production).

Ces étapes sont automatisées dans un script deploy.ps1, qui enchaîne une vérification de syntaxe PHP, le commit et le push Git, le déploiement fly.io, la migration si nécessaire, et quelques vérifications post-déploiement, pour éviter d'oublier une étape en cas de déploiement un peu précipité.